

Governança — CODEOWNERS, PR Template e Checklist de Auditoria (Aurora)

Objetivo

Completar a governança do repositório com **CODEOWNERS**, **template de Pull Request** e o **Checklist de Auditoria** da proteção da `main`, garantindo responsabilidade clara, revisão qualificada e rastreabilidade — com fundamentos, justificativas e arquivos prontos para uso.

1) CODEOWNERS — Fundamentos e Por Quê

O que é: arquivo especial em `/.github/CODEOWNERS` (ou na raiz) que mapeia **quem é responsável por revisar** alterações em caminhos específicos.

Benefícios (técnicos e pedagógicos): - **Responsabilidade explícita** por áreas do código (quem revisa o quê). - **Qualidade de revisão:** PRs passam por quem conhece o contexto. - **Fluxo profissional:** alunos vivenciam o processo real de ownership e revisão. - **Escalabilidade:** funciona bem em monorepo com vários microserviços.

Como funciona: - O **path mais específico** ganha prioridade. - Você pode apontar **times** (ex.: `@org/devs`) ou **usuários** (`@aluno1`). - Para **exigir** a aprovação de donos, ative depois a flag: *Require review from Code Owners* (já preparamos o comando mais adiante).

1.1 Arquivo pronto (substitua pelos seus times/usuários)

Crie `/.github/CODEOWNERS` com um destes **modelos**. Escolha **A (monorepo)** se você tem múltiplos serviços, ou **B (repo simples)** se for single-service.

A) Monorepo (exemplo por apps/packages):

```
# Donos por microserviço
/apps/users-service/**      @org/time-users @aluno1 @aluno2
/apps/events-service/**     @org/time-events
/packages/**                @org/time-platform

# Infra e automação
/.github/**                 @org/devops
/docker/**                  @org/devops

# Fallback (cuidado: pega o resto)
/*                           @org/time-platform
```

B) Repo simples (por áreas):

```
/src/**          @org/time-devs @aluno1 @aluno2
/test/**         @org/time-devs
/.github/**      @org/devops
/*              @org/time-devs
```

Importante: mantenha o **professor apenas com permissão Read** no repositório e **não o inclua** no CODEOWNERS (para não bloquear PRs). Ajuste `@org/...` e `@aluno...` para nomes reais de times/usuários.

1.2 Habilitar exigência de Code Owners (depois do arquivo existir)

```
# Exigir revisão de owners (mantém demais proteções já ativas)
gh api -X PUT -H "Accept: application/vnd.github+json"
  repos/:owner/:repo/branches/main/protection
  --input - <<'JSON'
{
  "required_status_checks": { "strict": true, "contexts":
["lint","build","test"] },
  "enforce_admins": true,
  "required_pull_request_reviews": {
    "required_approving_review_count": 1,
    "dismiss_stale_reviews": true,
    "require_last_push_approval": true,
    "require_code_owner_reviews": true
  },
  "required_conversation_resolution": true,
  "required_linear_history": true,
  "allow_force_pushes": false,
  "allow_deletions": false,
  "restrictions": null
}
JSON
```

Por quê: alinha **responsabilidade** → **aprovação**. A revisão passa por quem domina a área tocada no PR.

2) Template de Pull Request — Fundamentos e Por Quê

O que é: um guia padrão para descrever mudanças, riscos e evidências. Ajuda alunos e revisores a seguirem um checklist claro.

Benefícios: - **Clareza e contexto** na análise do PR. - **Disciplina de testes e documentação** antes do merge. - **Coleta de evidências** (prints do CI, links), útil para avaliação em aula.

2.1 Arquivo pronto (`.github/pull_request_template.md`)

```
## Contexto
<!-- Por que esta mudança é necessária? Referência a issue/tarefa. -->

## O que foi feito
<!-- Resumo objetivo das alterações (arquivos/chaves). -->

## Evidências
- [ ] CI verde (prints ou link para os jobs `lint`, `build`, `test`)
- [ ] Screenshots/Logs relevantes (se aplicável)

## Testes/Validações
- [ ] Testes unitários/e2e passaram
- [ ] Documentação atualizada (README/Swagger)
- [ ] Sem quebra de contrato público (API/DTOs)

## Riscos & Rollback
<!-- Riscos conhecidos e como reverter se necessário. -->

## Checklist de revisão
- [ ] Código segue padrões do projeto
- [ ] Nome de commits/PR descritivo
- [ ] Comentários de conversa resolvidos

## Revisores sugeridos
<!-- Marque time/owners conforme paths alterados (CODEOWNERS). -->
```

Por quê: torna explícito o que será avaliado no PR e treina para o mercado (qualidade, evidências, rollback).

3) Checklist de Auditoria — Proteção da `main` (para o README)

Inclua este bloco ao final do `README.md` do repo ou do microserviço:

```
## Auditoria – Proteção da `main`
- [x] Pull Request obrigatório
- [x] Status checks obrigatórios: lint, build, test (**strict: true**)
- [x] Branch up-to-date exigida
- [x] Enforce admins (sem bypass)
- [x] 1 aprovação + dismiss stale + last push approval
- [x] Conversation resolution + linear history
- [ ] CODEOWNERS ativo (arquivo presente)
- [ ] Require review from Code Owners habilitado

### Evidências anexadas
- Screenshot das regras de Branch Protection
```

- Link de PR com "All checks have passed" (lint/build/test)
- Execução do workflow CI (link dos jobs)

Justificativa: este checklist vira um **artefato de auditoria** para a disciplina (prova de qualidade e governança).

4) Passo a passo sugerido (commit único "governança")

```
# 1) Criar os arquivos
mkdir -p .github
cat > .github/CODEOWNERS <<'EOF'
/src/**                                @org/time-devs @aluno1 @aluno2
/test/**                              @org/time-devs
/.github/**                           @org/devops
/*                                    @org/time-devs
EOF

cat > .github/pull_request_template.md <<'EOF'
## Contexto
## O que foi feito
## Evidências
- [ ] CI verde (lint/build/test)
- [ ] Screenshots/Logs
## Testes/Validações
- [ ] Unit/E2E
- [ ] Docs
- [ ] Sem quebra de contrato
## Riscos & Rollback
## Checklist de revisão
- [ ] Padrões do projeto
- [ ] Nomes descritivos
- [ ] Conversas resolvidas
## Revisores sugeridos
EOF

# 2) Adicionar o checklist ao README (edição manual ou por editor)
# (Cole o bloco "Auditoria – Proteção da main" no final do README)

# 3) Commit e PR
git checkout -b chore/governanca-baseline
git add .github/CODEOWNERS .github/pull_request_template.md README.md
git commit -m
"chore: governança baseline (CODEOWNERS, PR template, auditoria README)"
git push -u origin chore/governanca-baseline
```

Abra o PR; **CI verde**; **aprovação**; **Squash and merge**.

Depois do merge: rode o PUT de proteção da `main` que **habilita Code Owners** (seção 1.2).

5) Verificações e Troubleshooting

5.1 Conferir proteção da branch (resumo)

```
gh api repos/:owner/:repo/branches/main/protection | jq '{
  strict: .required_status_checks.strict,
  contexts: .required_status_checks.contexts,
  reviews: .required_pull_request_reviews,
  enforce_admins: .enforce_admins.enabled,
  linear: .required_linear_history.enabled,
  conversations: .required_conversation_resolution.enabled
}'
```

5.2 Erros comuns

- **PR bloqueado após ativar Code Owners:** verifique se o **path** alterado tem um **owner** correspondente; ajuste o arquivo.
 - **Contexts não aparecem na UI:** rode a CI ao menos uma vez para registrar `lint/build/test`.
 - **404 ao usar** `:owner/:repo`: é placeholder; substitua por `owner/repo` ou execute dentro do repo.
-

6) Escalonando para múltiplos microserviços

- Crie entradas específicas por serviço (ex.: `/apps/users-service/**`, `/apps/payments-service/**`).
 - Mantenha um **time por domínio** (users, events, payments) para refletir ownership real.
 - Replique o **template de PR** e o **checklist de auditoria** nos READMEs de cada serviço.
-

Conclusão

Com **CODEOWNERS**, **PR template** e **auditoria no README**, a turma ganha **clareza de responsabilidades**, **disciplina de revisão** e **evidências objetivas** da proteção da `main`. Esses artefatos fecham o ciclo de governança iniciado com PR/CI e branch protection — e preparam o monorepo para crescer com segurança e previsibilidade.